Decision_making.py 주석

2021년 8월 27일 금요일 오전 12:56

```python
#!/usr/bin/env python
import sys, os
sys.path.append(os.path.dirname(os.path.abspath(os.path.dirname(__file__))))
from functions.CRI import CRI
import numpy as np
from math import pi, sqrt, cos, sin, atan2, asin
from numpy import deg2rad, rad2deg
import pandas as pd
import rospy

class decision_making:
    """충돌회피를 위한 의사 결정에 필요한 함수들이 정의됨"""
    def __init__(self):
        data_sheet = { 'RD': []}
        data_sheet2 = { 'isFarway': []}
        self.pre_distance = pd.DataFrame(data_sheet)
        self.pre_status = pd.DataFrame(data_sheet2)

    def is_Need_CA_analysis(self, TS, threshold, local_goal_EDA):
        '''1) 자선과 타선사이의 거리가 일정거리 이내인가?
           2) 선박조우상황이 'safe' or 'stand-on'인가?
           3) 현재 타선과 멀어지는 중인가?

           를 활용하여 충돌위험이 있는지를 판단함 '''
        TS_ID = list(map(int, TS['Ship_ID'].index))

        status = []
        for ts_ID in TS_ID:
            if TS.loc[ts_ID]['status'] == 'Head-on':
                if TS.loc[ts_ID]['RD'] ≤ 1.5 * threshold:
                    status.append(True)
                else :
                    status.append(False)
            elif TS.loc[ts_ID]['status'] == 'Overtaken':
                status.append(False)
            # elif TS.loc[ts_ID]['status'] == 'Port crossing':
            #     status.append(False)
            else:
                if TS.loc[ts_ID]['RD'] ≤ threshold:
                    if TS.loc[ts_ID]['status'] == 'Safe' or \
                            TS.loc[ts_ID]['isFarway'] == True :
                        status.append(False)
                    #<<<<<<<<<------------- 이 부분 재확인이 필요함!
                    elif local_goal_EDA < threshold:
                        if (TS.loc[ts_ID]['status'] == 'Overtaking' or TS.loc[ts_ID]['status'] == 'Overtaken') \
                                and  0 ≤ deg2rad(TS.loc[ts_ID]['local_rc'])< (7 * pi/8) :
                            status.append(False)
                        else:
                            status.append(True)
                    # elif (TS.loc[ts_ID]['status'] ≠ 'Starboard crossing') \
                    #           and  (3*pi/8) ≤ abs(deg2rad(TS.loc[ts_ID]['local_rc']))< (7* pi/8) :
                    #     status.append(False)
                        '''
                        elif (TS.loc[ts_ID]['status'] ≠ 'Starboard crossing') \
                                    and  (3*pi/8) ≤ abs(deg2rad(TS.loc[ts_ID]['local_rc']))< (7* pi/8) :
                                status.append(False)
                        ## 21.08.12 KRISO TEST 수행 중 발생한 이슈
                        # > 1) Head-on 으로 피하는 중에 port crossing 상황으로 인식
                        # > 2) Head-on & port crossing 사이의 threshold 값이 다름
                        # > 3) prot crossing 상황에서 모형시험을 위한 `local_rc`가 `false` case 에 들어와버림
                        # > 4) 그렇다고 해당 부분을 삭제할 수는 없는 것이, 충돌회피 수행후, global goal에 들어가기 위해서는 필요하다고 판단됨
                        '''


                    # 이 부분 재확인이 필요함! --------->>>>>>>>>>>>>>
                    else :
                        status.append(True)
                else :
                    status.append(False)

        TS['isNeedCA'] = status

        return TS

    def getting_farther_away(self, TS):
        """ 멀어지는 중인가를 판단함"""
        TS_ID = list(map(int, TS['Ship_ID'].index))

        if self.pre_distance.empty:
            for ts_ID in TS_ID:
                self.pre_distance.loc[ts_ID] = TS.loc[ts_ID]['RD']
                self.pre_status.loc[ts_ID] = [False]

        id = []
        for ts_ID in TS_ID:
            further_distance = self.pre_distance.loc[ts_ID]['RD'] - TS.loc[ts_ID]['RD']
            getting_father = f"is {ts_ID} moving further? : {further_distance}"
            rospy.loginfo(getting_father)

            if self.pre_distance.loc[ts_ID]['RD'] == TS.loc[ts_ID]['RD'] :
                status = self.pre_status.loc[ts_ID]['isFarway']
                id.append(status)
                self.pre_distance.loc[ts_ID] = TS.loc[ts_ID]['RD']

            elif (self.pre_distance.loc[ts_ID]['RD'] - TS.loc[ts_ID]['RD']) < -0.01:
                # print("Need global pathplanning") # 멀어지는 상황이면
                id.append(True)
```

· Head-on 의 경우, 다른 조우 상황 대비 1.5 배 먼곳에서부터 피할 예정.

타선도 COLREGs 규정을 따른다면, 주석 해제 바람.

보제 해결후 재 개재

Update rate 차이로 인해, 직전과 같은 값이 들어온 경우, 직전의 state로 놓여짐.

```python
            self.pre_distance.loc[ts_ID] = TS.loc[ts_ID]['RD']

        elif (self.pre_distance.loc[ts_ID]['RD'] - TS.loc[ts_ID]['RD']) < -0.01:
            # print("Need global pathplanning") # 멀어지는 상황이면
            id.append(True)
            self.pre_distance.loc[ts_ID] = TS.loc[ts_ID]['RD']
        else:
            id.append(False)
            self.pre_distance.loc[ts_ID] = TS.loc[ts_ID]['RD']

    TS['isFarway'] = id
    self.pre_status['isFarway'] = id

    return TS


def isNeedCA(self, TS):
    """ 충돌위험이 있다면, boolean으로 알려줌"""
    cond = (TS['isNeedCA'] == True)
    condition = TS[cond]
    if condition.empty:
        return False
    else:
        return True

# def vector2Heading(self, rad):
#     # rad = -rad
#     if rad ≥0:
#         return rad
#     elif rad<0:
#         return (rad +2 * pi) % (2*pi)

def vector2Heading(self, rad):
    rad = -rad
    if rad ≥0:
        return rad
    elif rad<0:
        return (rad +2 * pi) % (2*pi)

def True_bearing(self, Xot, Yot):
    '''
    `output`
        True bearing : 절대좌표계 기준 OS & TS 사이의 방위각 [deg]
    '''
    TB = self.vector2Heading(atan2(Yot, Xot))
    # TB = atan2(Xot, Yot)
    # # TB = self.angle_correction_0804(TB)
    # if TB < 0 :
    #     TB += 2*pi
    return rad2deg(TB)

def Relative_bearing(self, OS_pos, TS_pos, Co):
    '''
    `input`
        OS_pos : 자선의 (x, y) 좌표
        TS_pos : 타선의 (x, y) 좌표
        Co : 자선 Heading angle [deg]
    '''
    Xot = TS_pos[0] - OS_pos[0]
    Yot = TS_pos[1] - OS_pos[1]

    Tb = self.True_bearing(Xot, Yot)

    if Tb - Co ≥ 0:
        RB = Tb - Co
    elif Tb - Co < 0:
        RB = Tb - Co + 360 # [deg]

    return RB %360

def Relative_course(self, OS_Head, TS_Head):
    '''
    output
        : relative course between OS to TS [deg]
    '''
    RC = TS_Head - OS_Head
    if RC < 0 :
        RC += 360

    return RC %360

def encounter_classification(self, RC, RB):
    '''
    `input`
        RC : Relative speed heading direction (psi_OT) [deg]
        RB : Relative bearing (beta_OT) [deg]
    '''
    psi_OT = deg2rad(RC) % (2*pi)
    beta_OT = deg2rad(RB) % (2*pi)

    if (psi_OT > pi/2) and (beta_OT < 3*pi/2) and ( abs(beta_OT - psi_OT) < pi/2):
        return "Safe"
    else:
        if (7 * pi/8) ≤ psi_OT < (9 * pi/8):
            return "Head-on"
        elif (9 * pi/8) ≤ psi_OT < (13 * pi/8):
            return "Starboard crossing"
        elif (3 * pi/8) ≤ psi_OT < (7 * pi/8):
            return "Port crossing"
        else:
            psi_TO = (2*pi - psi_OT) % (2*pi)
            beta_TO = (pi + beta_OT - psi_OT) % (2*pi)

            if (5*pi/8) ≤ beta_OT < (11*pi/8):
                return "Overtaken"
            elif (5*pi/8) ≤ beta_TO < (11*pi/8):
                return "Overtaking"
```

직전의 state로 불러옴.

https://ieeexplore.ieee.org/document/9234094
(Efficient COLREG-Compliant Collision Avoidance in Multi-Ship Encounter Situations)

① 조우상황 여부 확인
(Situation analysis)

a) True → 다선이 이번에는 '자선에 대는 타선의 선수각도' 기준으로 고려함
b) False → 굳이 이로는 피하고 있는 친구

② 조우상황 분류
(Encounter classification)

a) HO
b) OT
c) Crossing
d) safe

③ 운항 결정
(maneuver decision)

a) way point Tracking
b) Stand-on

③ 운항 결정
(maneuver decision)

a) way point Tracking
b) Stand-on
c) Give-way

```python

            if (5*pi/8) ≤ beta_OT < (11*pi/8):
                return "Overtaken"
            elif (5*pi/8) ≤ beta_TO < (11*pi/8):
                return "Overtaking"
            elif beta_OT < (5*pi/8):
                return "Starboard crossing"
            else:
                return "Port crossing"


    def distance(self, pose1, pose2):
        """ compute Euclidean distance for 2D """

        return sqrt((pose1[0]-pose2[0])**2+(pose1[1]-pose2[1])**2) + 0.0001 # = sqrt(np.dot(pose1, pose2))
```